

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	V. YESHWANTH	Reg. No.:	192511300
Section / Batch:		Date:	17/07/2026

Code of Conduct

I, V. Yeshwanth (192511300) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V. Yeshwanth

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

			are dequeued k) I is Dequeued. Demonstrate with a neat sketch of Front, Rear and count in a queue and circular queue. 2. Show the linked list configuration resulting from each series of list operations using the List ADT. Assume that lists L1 and L2 are empty at the beginning of each series. Show where the current position is in the list. (a) L1.append(10); L1.append(20); L1.append(15); (b) L2.append(10); L2.append(20); L2.append(15); L2.moveToStart(); L2.insert(39); L2.next(); L2.insert(12);
35	192511435	SRI KRISHNA P	1. Convert the following infix expression into postfix notation. $((A+B)*(C-(D/E))+F)*(G-H)$. Also determine the maximum stack size required during the conversion. 2. A circular parking system has 6 parking slots. <i>Operations:</i> ➤ Car1 enters ➤ Car2 enters ➤ Car3 enters ➤ Car1 exits ➤ Car4 enters ➤ Car5 enters ➤ Car6 enters ➤ Car7 enters <i>Questions:</i> ➤ Draw the circular queue after each operation. ➤ Identify the positions of the front and rear pointers. ➤ Explain why a circular queue is more efficient than a linear queue in this scenario.
36	192521480	SRIVATSAN L	1. Given a singly linked list L: L1-> L2->L3...-> Ln-1->Ln, reorder and print it to: L1-> Ln ->L2 -> Ln-1. 2. Insert the following elements 150, 250, 350, 450, 550 into the double linked list and delete the elements 150 and 250 from the double linked list. Show the linked structure using a node diagram before and after delete operations.
37	192511300	V YESHWANTH	1. A circular queue has a size of 6. Execute the following operations: Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Enqueue(40), Enqueue(50), Enqueue(60), Enqueue(70). Draw the queue after each operation and identify the values of the front and rear pointers. 2. A router processes packets using a queue. Packets arrive in the order: P1, P2, P3, P4, P5 The buffer size is 4 packets. <i>Questions:</i> ➤ Identify which packet is dropped. ➤ Explain why the packet is discarded. ➤ Suggest a better queue management strategy to reduce packet loss.

38	192511310	VAIBHAVAK LAKSHMI K	- Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. - A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192524458	VASANTHAN S	- Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. - A queue implemented using an array report "Queue Full" even though some positions are empty. - Analyze why this occurs. - Explain how a circular queue solves this problem.
40	192524083	YARRAPPAREDDY ARI SAI GOPAL REDDY	- Consider the singly linked list: 10 → 20 → 30 → 40 → 50 Delete the node containing 30. Explain how the links change after deletion. - Given two sorted linked lists: L1: 1 → 3 → 5 → 7 L2: 2 → 4 → 6 → 8

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Circular Queue Operations:

→ A Circular queue is a linear data structure that follows the FIFO principle, but the last position is connected back to the first position to form a circle. This allows efficient use of space, avoiding the wastage seen in a normal linear queue.

Formula:

$$\Rightarrow \text{Enqueue: } \text{Rear} = (\text{rear} + 1) \% \text{Size}$$

$$\Rightarrow \text{Dequeue: } \text{front} = (\text{front} + 1) \% \text{Size}$$

$$\Rightarrow \text{Full Condition: } (\text{rear} + 1) \% \text{size} = \text{front}$$

$$\Rightarrow \text{Empty Condition } \text{front} = -1$$

Step-by-step Execution:-

1) Enqueue (10)

* Queue was empty, set $\text{front} = 0$;

$$\text{rear} = (-1 + 1) \% 6 = 0; \text{ insert } 10$$

* Queue state (index 0-5)

[10, -, -, -, -, -]

* $\text{front} = 0$

* $\text{rear} = 0$

2) Enqueue (20)

$$\Rightarrow \text{rear} = (0+1) \% 6 = 1; \text{insert } 20$$

$$\Rightarrow [10, 20, -, -, -, -]$$

$$\Rightarrow \text{front} = 0 \text{ and } \text{rear} = 0$$

3) Enqueue (30)

$$\Rightarrow \text{Rear} = (1+1) \% 6 = 2; \text{insert } 30$$

$$\Rightarrow [10, 20, 30, -, -, -]$$

$$\Rightarrow \text{front} = 0 \text{ and } \text{rear} = 2$$

4) Dequeue()

$$\Rightarrow \text{Remove value at front (10);}$$

$$\text{front} = (0+1) \% 6 = 1$$

$$\Rightarrow [-, 20, 30, -, -, -]$$

$$\Rightarrow \text{front} = 1 \text{ and } \text{rear} = 2$$

5) Enqueue (40)

$$\Rightarrow \text{rear} = (2+1) \% 6 = 3; \text{insert } 40$$

$$\Rightarrow [-, 20, 30, 40, -, -]$$

$$\Rightarrow \text{front} = 1 \text{ and } \text{rear} = 3$$

6) Enqueue (50)

$$\Rightarrow \text{rear} = (3+1) \% 6 = 4; \text{insert } 50$$

$$\Rightarrow [-, 20, 30, 40, 50, -]$$

$$\Rightarrow \text{front} = 1 \text{ and } \text{rear} = 4$$

Enqueue(60)

$\Rightarrow \text{Rear} = (4+1) \% 6 = 5; \text{insert } 60$

$\Rightarrow [-, 20, 30, 40, 50, 60]$

$\Rightarrow \text{front} = 1 \text{ and } \text{rear} = 5$

8) Enqueue(70)

$\Rightarrow \text{check full: } (5+1) \% 6 = 0 \neq \text{front}(1) \rightarrow \text{not full;}$

$\text{rear} = 0; \text{insert } 70$

$\Rightarrow [70, 20, 30, 40, 50, 60]$

$\Rightarrow \text{front} = 1 \text{ and } \text{rear} = 0,$

Final Diagram:

index:	0	1	2	3	4	5
value	70	20	30	40	50	60

Conclusion:

After all operations : $\text{front} = 1$ (pointing to value 20), $\text{rear} = 0$ (pointing to value 70). The queue now holds 6 elements and is full, since rear wrapped around from index 5 back to index 0 - demonstrating the key advantage of a circular queue over a linear queue (reuse of freed space)

Router packet Queue

Given:

- * Buffer size = 4 packets
- * Arrival order = P_1, P_2, P_3, P_4, P_5
- * Queue follows FIFO discipline

Step-by-step execution:-

Step	Packet Arrives	Buffer occupancy	Buffer contents	Result
1	P_1	1/4	P_1	Accepted
2	P_2	2/4	P_1, P_2	Accepted
3	P_3	3/4	P_1, P_2, P_3	Accepted
4	P_4	4/4	P_1, P_2, P_3, P_4	Accepted (buffer now full)
5	P_5	4/4 (full)	P_1, P_2, P_3, P_4	Dropped (No space)

a) Which packet is dropped — P_5 is dropped

b) Discarded:

Since none of the earlier packets ($P_1 - P_4$) have been dequeued by the router before P_5 arrives the buffer remains at its maximum capacity of 4. When P_5 arrives, there is no free slot to enqueue it, so it is discarded. This condition is known as buffer overflow or tail drop; a common occurrence in FIFO queues with fixed, finite size.

c) Better Queue Management Strategies:

⇒ Increase buffer size:

* Provides more room to absorb traffic bursts

⇒ RED:

* Drops packets probabilistically before buffer is full, signaling senders to slow down

Priority Queuing (Qos):

Important packets get priority - less critical ones are dropped first.

Congestion Control:

Sender reduces transmission rate when congestion is detected, preventing overflow.

Multiple queues per flow:

Prevents a single heavy traffic flow from filling the entire buffer.

Best recommended approach:

A combination of RED with priority queuing this reduces packet loss more effectively than simply increasing buffer size, which can introduce latency (bufferbloat).

Wong